本章路線

這章要回答的核心問題是：在身分、程式與網路都可能被攻擊的環境中，哪些 cryptographic 與 system mechanisms 能提供可驗證的安全保證？

前置知識。 protection、網路基本概念、機率與雜湊概念。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

- 區分 confidentiality、integrity、availability、authentication
- 分析對稱/非對稱加密與 hash 用途
- 判斷防禦措施在攻擊模型下的實際覆蓋

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.1 Security problem

- 固定威脅模型與CIA目標，分清身分、授權與新鮮性。
- 訊息有效與授權新鮮性不同，重放需要receiver state處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先定資產、對手能力與安全目標

confidentiality限制未授權讀取；integrity限制未授權變更；availability要求授權服務能在規定條件下運作。威脅是可能破壞目標的條件，vulnerability是可被利用的弱點，attack是實際嘗試。三個目標可能同時受損：竊取後再加密刪除資料不只是一種分類。

原創情境：成績服務保護成績、帳密與可用性。對手可控制網路、提交任意表單，但不能直接改核心或取得伺服器私鑰。若後來假設私鑰已被偷，原來的網路認證推論就不能沿用。安全結論必須連同對手能力、信任根、金鑰與端點完整性前提一起讀。

physical、human、OS、network都會影響此邊界。TLS保護線路，不保護已被鍵盤側錄程式取得的輸入；ACL限制未授權使用，不保證持有效帳號的惡意操作符合業務規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Masquerading、重放與中間人不是同一問題

masquerading冒用身分；replay重送曾有效的訊息；MITM同時控制兩端對彼此的觀察。來源IP不是密碼學身分，使用一個公鑰也不代表該key真的屬於目標服務。

例：有效訊息 `transfer(A,B,100,id17)` 帶正確MAC。對手看不懂內容仍可能原封重送；驗MAC成功只證明某持key者曾產生該內容，不證明它是新的一次授權。需要把session/sequence/nonce/操作id納入受驗資料，並由接收方維護已使用狀態；若重試要求回舊結果，需持久化去重與交易原子性。

安全目標應寫為「未授權者不能偽造新有效操作」與「同一授權不被重複執行」，而不是籠統說「用了加密就安全」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.2 Program threats

- 程式權限與感染/觸發/隱藏方式分開。
- 以明確byte容量檢查越界與防禦保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

程式威脅：執行權限與傳播方式分開

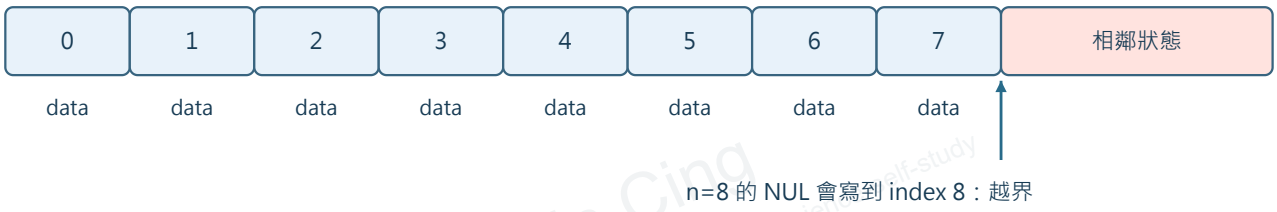
Trojan horse以看似正當功能濫用被執行時的權限；backdoor/trap door繞過正常入口；logic bomb由特定條件啟動。virus需依附並感染host檔/程式，worm可透過服務/網路自我傳播；分類可重疊，不表示病毒只可能出現在某一家OS。

PATH含不可信工作目錄時，同名工具可能取代預期程式；受害者執行後攻擊程式取得的是受害程式實際持有的權限，不會因稱作Trojan就自動取得root。編譯器本身若被植入惡意行為，僅檢查應用source不足，因為產物不一定對應source。

保護邊界包含可執行檔、loader、compiler與更新來源。檔案/boot/macro/source virus描述感染位置；polymorphic/encrypted/stealth/armored描述規避分析方法；multipartite可感染多位置。辨識byte特徵與判定執行行為互補，不能據未命中特徵斷言無感染。

Buffer overflow：先證明界限，再談控制流

抽象 byte 配置：字串終止字元也占一格



容量 8 → 完整 C 字串最多 7 data bytes + 1 NUL；binary buffer 改用顯式長度。

固定byte buffer容量8，合法indices0–7。C字串若資料長n，還需一個NUL，因此完整複製的前提n+1≤8。n=8時資料恰占0–7，NUL落index8已越界；不是「小於等於8字元都安全」。實際stack布局由ABI/compiler決定，圖只示範相鄰狀態可能被破壞，不提供特定平台位址。

Buffer overflow：先證明界限，再談控制流 （續）

安全copy有兩種不同契約：reject過長，或明確截斷。若截斷，取k=min(n,7)，copy前k bytes，再寫buf[k]=0；只做`strncpy(...,7)`未必自動補尾端NUL。若是binary資料，不應用strlen判長，而要攜帶顯式長度並檢查範圍與整數溢位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多層防禦的保證各不相同

bounds/type/lifetime檢查針對非法記憶體操作的成因；stack canary嘗試在返回前偵測部分覆寫；NX/W⊕X限制資料頁執行；ASLR使地址較難預測；control-flow約束限制可接受目標。NX不阻止越界寫，也不能單獨排除重用既有可執行程式片段的攻擊。

即使沒有改return address，覆寫相鄰的`is_admin`、length或object pointer也可能破壞授權。故「攻擊沒有執行stack上的code」不等於安全。最小權限再限制已受害component可造成的損害，仍不能替代修正越界來源。

受管理語言的safe subset可消除某些記憶體錯誤類型，但unsafe/native code、runtime錯誤及業務授權仍是邊界；不得把「type-safe」解讀成任何安全目標都已證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.3 System and network threats

- 服務暴露、worm傳播與DoS皆可由資源/狀態模型分析。
- pending-state配置率與保留時間共同決定DoS占用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worm 傳播：局部漏洞與全網負載的連鎖

歷史案例Morris worm（1988）同時利用服務漏洞與遠端信任關係，感染後再尋找可達主機。它說明單點弱點能跨機器放大，也說明同一機器重複感染會消耗資源；不是只有竊取資料才算安全事件。

原創離散模型：每一輪每個感染host最多感染2個尚未感染host、無重複、無清除；$I_0=1$，$I_{t+1}=3I_t$，因此I=[1,3,9,27]。這是初期理想上界模型，不是實測Morris數據。若100台封閉網路，必須改$I_{t+1}\le\min(100,3I_t)$，不能繼續算到243台。

port scanning提供可達服務的觀察，開放port本身不等於存在漏洞，banner也不必可信。授權檢查與弱點修補減少attack surface；關閉不需要的服務只移除該入口，不修復其他允許服務的輸入錯誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DoS：資源守恆與防禦副作用

教學服務有1000個pending slots，每次未完成握手保留10 s；若無提前清理，平均到達率100/s即可占滿1000（以穩態L=λW計）。攻擊不必取得登入權，只要能迫使伺服器配置稀缺狀態。cookie/限額/延後配置可改資源模型，但不能讓上游鏈路容量無限大。

合法工作80/s、攻擊40/s，服務上限100/s且無區分，至少20/s工作無法在穩態消化。rate limiting必須指定按user/IP/tenant/全域哪個key；共用NAT下按IP會連帶影響合法者。

帳號連錯5次鎖10分鐘可阻止單帳號猜密碼，也可被用來惡意鎖住他人。DDoS是多來源協作，flash crowd是合法需求暴增；只看負載高不足以判定惡意來源，回應必須兼顧可用性與誤封代價。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.4 Cryptography

- 每個primitive只提供指定保證；key binding與freshness另證。
- RSA全輸入正確性、nonce重用與歷史TLS握手逐步推導。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

密碼學工具：目標、key 與可驗證性

工具	保密	完整性/來源	誰能產生與驗證
對稱加密	適當scheme下提供	單純加密不保證	持同secret key者
public-key encryption	適當scheme下提供	不直接證sender	任何人可encrypt，private key可decrypt
hash	不提供	只偵測與可信digest不符	任何人可計算
MAC	不提供	持key者來源/完整性	持同key者可產生及驗證
digital signature	不提供	private signer來源/完整性	private產生、public驗證

正確性是$D_k(E_k(m))=m$；安全性是特定對手模型下難以區分/偽造等計算性保證，並非「不知道key在邏輯上不可能猜中m」。演算法公開，key保密；公開key不需保密，但其身分綁定需要完整性/認證。

Symmetric cipher：nonce 重用會洩漏什麼

stream模型$c=m\oplus s$、$m=c\oplus s$。例m_1=0x41、m_2=0x42、s=0xA6，得到c_1=0xE7、c_2=0xE4；同keystream重用時$c_1\oplus c_2=0x03=m_1\oplus m_2$。已知$m_1$即可求s=0xA6，再解$m_2$。這是代數恆等式，不需破解底層cipher。

block cipher處理固定長度block，長訊息還需正確mode與nonce/IV規則。AES block128 bits、keys128/192/256 bits是不同參數，不能互換。DES/3DES/RC4/WEP為原書歷史脈絡；本章不提供把它們部署為新系統的建議。

保密與驗證應由完整受分析的scheme提供；自製「加密後再附公開hash」仍不能阻止對手改ciphertext並重算未受保護hash。

RSA 原創小數例：完整模冪

只示範數論，非安全加密：p=5、q=11、n=55、φ(n)=40，e=3，d=27，ed=81=1+2φ(n)。public=(55,3)，private=(55,27)。m=12：12^2 mod55=34；12^3 mod55=34×12 mod55=23，所以c=23。

解密平方表：23^1→23，23^2→34，23^4→1，23^8→1，23^{16}→1；27=16+8+2+1，所以23^{27} mod55=1×1×34×23 mod55=12。不能只說「用私鑰解回」而省掉mod值。

raw RSA確定性且具代數結構，不是可直接部署的encryption/signature scheme；實際編碼/padding/金鑰長度與用途分離屬協定規格。本例小質數沒有任何保密強度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA 正確性：包含不互質的明文

對任意m∈{0,⋯,n−1}，因ed=1+k(p−1)(q−1)。模p：若p ∣ m，m^{ed}≡0≡m；若p∤m，Fermat給m^{p-1}≡1，因此m^{ed}=m$(m^{p-1})^{k(q-1)}$≡m。模q同理。p、q相異質數，CRT推出m^{ed}≡m modpq。

這證明decrypt(encrypt(m))的數學正確性，不證明RSA假設、抗chosen-ciphertext安全或實作無side channel。只對gcd(m,n)=1援Euler會漏掉可被p或q整除的合法m；上面分支補齊邊界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hash、MAC 與 signature 的差別

n-bit digest只有2^n種輸出，長訊息空間更大，collision由鴿籠原理必存在。collision resistance指有效找任意$m \neq m'$且$H(m)=H(m')$很難；second-preimage指給定m再找m'很難，兩者不同。digest相等是計算性證據，不是$m=m'$的數學定理。

公開傳$(m,H(m))$，對手可改成$(m',H(m'))$，所以沒有來源認證。MAC使用secret key，驗證者也能產生MAC，因此不能單靠MAC向第三方證「只有另一方簽過」。signature由private key產生、public驗證；仍需可信public-key binding、完整訊息編碼與key保護，不能把簽章抽象成「任意加密反過來用」。

HMAC是帶key的標準構造，不等同隨意$H(key \mathbin{/\!/} m)$。認證內容應包括context、角色、session與序號，避免跨用途搬用一個有效tag。

Key distribution 與中間人

公鑰不必保密，但不能讓對手任意替換身分綁定



未驗綁定：A 用 K_M 加密 → M 能解；cipher 正確，對端身分仍錯。

可信 certificate chain + 名稱 / 用途 / 有效性 + 握手證明持 private key，才能建立相應身分。

A欲連B，若只從網路收「我是B，公鑰K」，M可改成K_M；A用K_M加密，M可解。加密演算法完全正確，身分卻錯。certificate把subject/service identity與key綁定，CA簽章需沿可信鏈驗證，另查名稱、用途、有效期與相關撤銷政策；不能只看certificate存在。

Key distribution 與中間人 (續)

共享對稱key若每對users各一把，N人需N(N−1)/2對；N=6為15，新增第7人需另6把。KDC可減少每個user長期key數，但把信任集中到KDC。public-key機制降低secret配送需求，仍不消除public-key認證與private-key保護。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

放在哪一層：endpoints 決定可見範圍

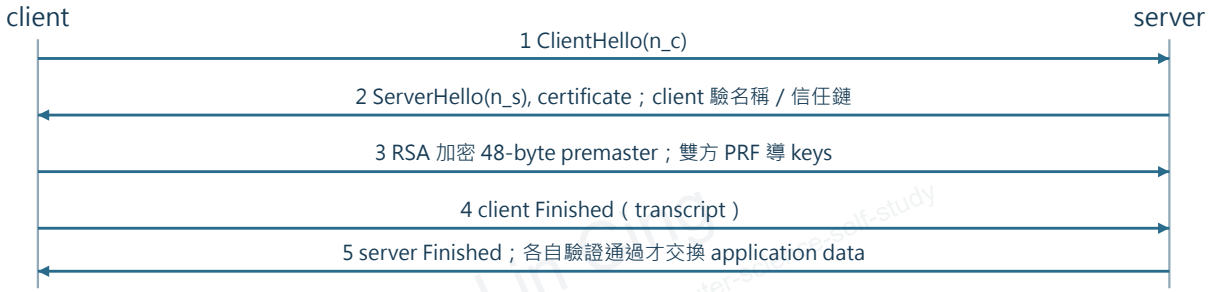
IPsec可保護兩個network endpoints間的IP流量；TLS保護其session endpoints間的資料；mail內容加密可跨越多個store-and-forward hops。若TLS在reverse proxy終止，proxy到backend的另一段不是自動被第一段TLS覆蓋；proxy可見解密內容。

link/network層保護不直接證明最終登入者的身分；應用認證仍需獨立契約。disk encryption保護離線媒體，不防已登入且有read權的程式讀明文。安全層數不能相加成無條件安全；每層要列誰持key、誰看明文、誰是TCB。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

歷史 TLS 1.2 RSA handshake 的必要步驟

歷史 TLS 1.2 RSA full handshake（省略格式 / alert / CCS 細節）



以下限於RFC5246的full handshake、RSA key exchange示意，不是TLS1.3：ClientHello帶n_c；ServerHello帶n_s與certificate；client驗可信鏈/目標名稱/用途/時間，再建立48-byte premaster secret（2-byte版本+46-byte隨機值），用規定的RSA編碼加密傳給server。雙方以PRF導出master與雙向traffic keys。

歷史 TLS 1.2 RSA handshake 的必要步驟 (續)

雙方還必須驗Finished對先前handshake transcript的認證；收到certificate只驗了「某CA簽這個key」，尚未證明當下對端持private key，也未驗整段握手沒被改。歷史RSA key exchange即使session後刪除traffic keys，若static RSA private key日後外洩且錄有handshake，仍可能解回舊premaster，不能因此宣稱forward secrecy。

SSL3與TLS版本不能當同義詞；RFC7568於2015年禁止使用SSL3。本節保留原書的協定教學目標，但精確限定版本與省略範圍。

認證與防重放：nonce、sequence 與原子提交

原創協定：server發一次性nonce n=42，pending[n]=(U,expiry)；client回 MAC_k("login"∥U∥n∥serverID)。server在同一原子操作中驗tag、驗pending/expiry，並把n改used。第一次可接受；第二次tag仍正確，但n已used所以拒絕。

推理： 正確MAC不能阻止複製舊tag；nonce狀態才能區分舊授權。只有驗證成功後才消耗nonce，且check與consume必須原子，否則兩個並行replay可同時看到pending。隨機nonce需要足夠熵及不重用；counter需跨重啟保存/換session。若是付款等操作，nonce consume與業務效果還要用交易或可恢復去重狀態連結，才不會crash後遺失/重做。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.5 User authentication

- password/OTP/biometrics的錯誤與猜測成本有各自分母。
- salt隔開每筆計算，不阻止offline guessing。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

User authentication 與 session 是兩個邊界

knowledge、possession、inherence分別是知道、持有與生物特徵。兩個password仍是同一類factor；MFA指兩類或以上，並非一定要三類。認證成功後產生的session token若被偷，後續請求可能繞過原先factor，因此session保護、失效與重新認證仍重要。

password verifier不應是明文。抽象記錄為{user,salt,cost,verifier=KDF(password,salt,cost)}；登入時用同salt/cost重算並比較。salt可公開，讓不同記錄不能共用同一個預算好的字典表；不阻止對特定記錄猜測，也不保證相同password在數學上永不hash collision。verifier外洩仍是offline guessing風險，存取控制仍需保護。

歷史UNIX八字元限制只適用特定舊scheme，不能當成所有UNIX規則；本章不把舊password政策當現代部署指引。

猜密碼的分母與成本

固定均勻隨機四位**十進位PIN**，包括0000，共10^4=10000種。無重複、無先驗偏差的猜測順序，平均排名(10000+1)/2=5000.5；每次1 ms則平均5.0005 s。這不是任意四字元password的空間，也不適用於人選偏態字典。

有1000筆不同salt記錄，字典100000候選：攻擊者不能只算100000個hash就與所有salt直接比較；逐record約需1000×100000次KDF。若cost讓每次5 ms，單工作單元總500000 s≈5.79天；可平行的資源、記憶體限制與已知偏態會改成本，不能把單核時間直接當實際攻擊時間。

online rate limit不會自動約束偷走verifier後的offline guesses；強迫鎖帳號也可能形成DoS。防禦需要分清攻擊在哪個邊界。

OTP 與 biometrics 的誤差模型

OTP/challenge-response把新鮮性加入認證。一次性是server只接受一次的狀態規則，不是畫面顯示不同數字就成立；TOTP的時間窗需處理已用碼/重放範圍，且可被即時釣魚轉送，不能自動推得phishing resistance。

biometric以threshold判matching score，false accept rate (FAR)以impostor attempts為分母，false reject rate (FRR)以genuine attempts為分母。教學資料：100000次冒用，FAR0.1%意味100次誤接受；10000次本人，FRR2%意味200次誤拒絕。提高threshold通常改兩種錯誤取捨；數字依測試人口/裝置/環境而定。

生物特徵不是可隨意更換的secret；template保護、活體/攻擊偵測、遺失設備與fallback政策都影響安全。多factor若共用同一已受害端點/復原流程，失敗風險可能相關，不能把兩個失敗機率無條件相乘。

15.6 Implementing security defenses

- 防禦要連到攻擊路徑；IDS以Bayes檢查告警解讀。
- baseline、checker與logs的完整性也是信任根。

政策、弱點評估與 defense in depth

政策指定資產、允許操作、責任與處置；assessment檢查configuration/程式/暴露服務是否符合，penetration test在授權模型下檢驗可利用路徑。通過一次scan只支持該時間、scope、工具與已知檢查範圍，不能證明無漏洞。

defense in depth要避免共同失效：兩層都依賴同一被偷key或同一個錯誤身分標頭，不代表獨立兩層。patch、least privilege、輸入/記憶體安全、segmentation、認證、監控與回復分別降低不同失效的機率/損害；不能只增加工具名稱而不指定誰阻止哪條攻擊路徑。

原創風險模型：單次損失10萬元、每年事件機率0.02，期望損失2000元/年；控制若把機率降到0.005則剩500元，期望減少1500元。它不描述尾端災難、相關事件或法規義務，不能只按平均值決定所有安全政策。

IDS／IPS：base rate 不能省略

signature辨識已編碼的可疑特徵；anomaly辨識偏離baseline；前者不保證所有未知攻擊都漏、後者不保證所有異常都惡意。IDS通常告警，IPS可介入阻擋；阻擋錯誤會直接影響availability。

原創資料：1000000事件中100是真攻擊；TPR=0.9，FPR=0.001。TP=90、FN=10；normal=999900，因此FP=999.9（期望）、TN=998900.1。P(attack|alarm)=90/(90+999.9)≈8.26%，不是90%。TPR的分母是真攻擊，precision的分母是全部告警。

Bayes式：precision=π・TPR/[π・TPR+(1−π)・FPR]。固定π=0.0001、TPR0.9，若要precision≥0.5，需FPR≤0.000090009…。baseline被入侵資料污染、概念漂移與相關事件聚合均會影響估計，不能拿開發資料回稱獨立驗收。

Integrity monitoring、antivirus 與可恢復性

可信baseline記錄檔案內容digest與metadata，重新比較可發現變動；baseline與checker若同被攻擊者修改，對比就失去根。hash matching的前提是使用適當抗collision算法與受保護原始digest，並非絕對同一內容的數學保證。

Tripwire式監控要分開authorized update與unexpected change；頻繁變動log不能因「本來會變」而完全放棄audit，要使用適當遠端/append-only/受保護記錄與關聯分析。honeypot用於觀察，不得成為可任意攻擊內網的新跳板。

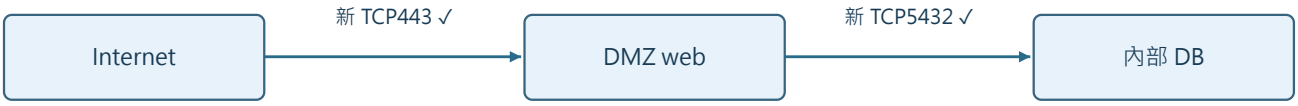
antivirus特徵、行為、sandbox各有偵測範圍；clean scan不證明合法供應鏈產物無惡意。發現問題後還需隔離、保全證據、修補來源、由可信備份恢復及輪換受影響credentials。audit record應帶subject/object/action/decision/time/correlation id，並保護其完整性與可用性。

15.7 Firewalling

- 以zone、direction、port、state推導first-match規則。
- 允許服務中的漏洞仍可穿過網路filter。

Firewall：先畫允許的連線，才寫規則

安全區不是全信任：只放行列明的方向與服務



Internet → DB 新連線：拒；web → DB22：拒；返回封包另依既存 state 驗證。

web 被攻陷後仍有 DB5432 這條必要通道；DB 必須另做認證與最小查詢授權。

原創stateful policy：Internet→DMZ web TCP443允許；DMZ web→內部DB TCP5432允許；內部→Internet TCP443允許；其餘**新連線**default deny；只讓符合既存state的返回封包通過。沒有一條「DMZ可去所有內網」規則。即使web被攻陷，對DB5432的既有必要權仍可被濫用，所以DB應另做認證/查詢權限。

Firewall：先畫允許的連線，才寫規則 (續)

stateless filter看每包header；stateful追connection state；application proxy可終止並理解協定；host firewall限制本機入口/出口。TLS未在proxy終止時，單純packet filter看不見加密payload的應用語義；允許443不代表443內所有內容安全。

規則求值與邊界

規則2–4分別實現Internet→web、web→DB及內部→Internet三種新連線；規則1處理符合既存允許連線state的雙向後續封包。TCP服務比對的是目的port。

次序	比對條件	動作
1	符合既存允許連線state的後續封包，雙向均可	允許
2	新TCP，Internet→DMZ web，目的443	允許
3	新TCP，DMZ web→內部DB，目的5432	允許
4	新TCP，內部→Internet，目的443	允許
5	其餘封包	拒絕

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

規則求值與邊界 （續）

封包情境	第一條命中規則	結果
新Internet→web TCP443	2	允許
新web→DB TCP5432	3	允許
新內部→Internet TCP443	4	允許
新Internet→DB TCP5432	5	拒絕
新web→DB TCP22	5	拒絕
DB→web，且確實符合既存連線的返回state	1	允許
web→DB，既存允許連線中的後續資料	1	允許

source/destination IP、zone、目的port、direction、state都要比對，不能只看到5432或ACK bit就放行。只宣稱established而沒有state-table證據的封包不命中規則1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

規則求值與邊界 （續）

如果把「deny all」放第1且仍採first-match，其他新連線規則不可達。真產品可能有不同chain/priority語義，須按其規格；此模型只是清楚展示規則順序與狀態的作用。

firewall不修復允許協定中的程式漏洞，不保證host身分只靠IP就可信，也無法在上游鏈路已塞滿時憑本地drop恢復頻寬。tunneling/出口控制與DNS等依賴需另列，不得以有DMZ圖就宣稱內網不可達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.8 Computer-security classifications

- TCSEC是受評配置/政策下的歷史功能與assurance要求。
- TCB強制指定政策，assurance不等於所有實際業務都已證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

歷史 TCSEC：功能要求與 assurance 分開

固定為美國DoD歷史TCSEC/Orange Book分類，不是今日所有產品的一條通用安全分數。D為未達較高類別；C1/C2重視discretionary access與accountability，C2加個體控制、audit與object reuse等要求；B1–B3逐步加mandatory labels、較完整的結構/分析要求；A1在B3功能上增加形式化設計/驗證與開發管理保證。

B2的covert-channel要求是識別/分析可能繞過正常資訊流的通道，不是「支援秘密外洩通道」。TCB是支撐政策強制執行的硬體/軟體/韌體集合；不在TCB的component錯誤不應破壞該政策，否則邊界畫錯。

認證結論限制在受評版本、配置、政策與環境。A1不代表業務永不出錯、所有攻擊都不可能或今日任何新版本自動繼承同保證；形式化證明也只對其模型、假設與規格成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.9 Windows 7 example

- 歷史Windows7 token、ordered DACL、MIC與UAC分層追蹤。
- DACL的remaining mask與MIC的operation-specific限制各有分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Windows 7：token、security descriptor 與 operation

固定歷史Windows7核心模型。process primary token帶user/group SIDs及privileges；server thread impersonation時可使用client context。object security descriptor含owner、DACL與SACL；DACL決定允許/拒絕，SACL可含audit與mandatory integrity label。已取得handle的granted access與後續操作也要對照，不是每個API都重跑同一open流程。

下例限制為普通specific access bits，無owner/privilege例外、restricted-token附加檢查或object ACE；用來逐步演示DACL：remaining=requested；依ACE順序，matching deny若與remaining相交則拒絕；matching allow移除其granted bits；remaining=∅可成功；掃完仍非空則拒。不是先把所有allow union後再把deny全部相減的通用算法。

DACL 的實際 mask trace

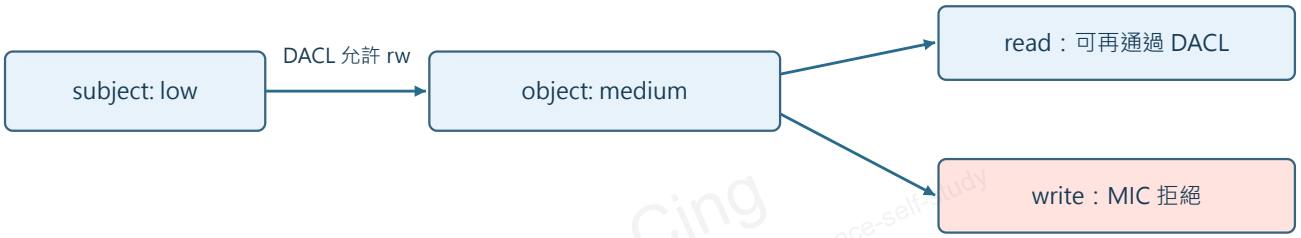
request={read,write}，token含user=U與enabled group=Students。DACL依序：deny(Students,write)；allow(U,{read,write})。初始remaining={r,w}；第一ACE命中且交集={w}，立即拒。若U不是Students，第一ACE略過，第二移除r,w，remaining空則允許。

若故意使用非canonical順序把allow(U,rw)放前，會先滿足全部請求而成功；所以不能說「任何位置的deny永遠覆蓋所有allow」。系統偏好的canonical順序將explicit deny置於相關allow前，inheritance等細節還需額外排序規則。

NULL DACL（無DACL）與存在但零ACE的empty DACL不同：前者在本普通access-check模型允許，後者沒有grant而拒絕。兩者都不表示可繞過另一層mandatory policy或特別privilege規則。

MIC 與 UAC：不是低完整性一律不能讀高物件

同一 DACL grant，mandatory policy 仍分 read / write



前提：mandatory label 只有 NO_WRITE_UP；若加 NO_READ_UP，read 分支也拒絕。

low subject對medium object，DACL允許r,w。若mandatory policy只有NO_WRITE_UP，write仍拒絕，read可在DACL與其他檢查通過後允許；若另設NO_READ_UP，read也拒。integrity主要保護不被低信任來源修改，不等於confidentiality clearance的完整模型。

MIC 與 UAC：不是低完整性一律不能讀高物件 （續）

Windows7 UAC下管理員的一般程式可使用filtered medium token；經受控elevation才取得相應較高token。UAC提示不是給任意程式創造使用者從未有的所有權限，也不能把UI確認本身當作惡意程式辨識器。

container inheritance與copy/move/既存object傳播依檔案系統/API選項而異；建立object時需檢查最終descriptor，不能假設更改parent ACL必自動以同方式重寫所有已開handle。

可組合的安全推理

認證綁定key或credential到subject；授權限制subject的operation；完整性/保密scheme保護訊息；freshness與receiver state防重放；transaction/recovery把授權連到一次可恢復效果。缺任一層都不能用另一層的名稱補上。

實際例：正確MAC的重放仍需拒；合法TLS但proxy已被攻陷不保證end-to-end秘密；DACL允許write仍可能被MIC拒；NX啟用但越界改資料仍可能破壞授權。結論應列前提、失效後果與可觀察驗證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一般證明、數值例與配置證據

RSA以Fermat/CRT證全域正確性，模冪表只核一個輸入；重用keystream的洩漏由XOR恆等式推出，不是靠挑某個cipher破解；IDS以Bayes連接不同分母，不能由單一TPR推precision。

歷史協定/OS案例需保留版本、endpoint與省略範圍。測試一次nonce重放、跑一個DACL trace或scan一次網路只支持相應觀察；不替代對所有state transition、威脅模型與安全規格的論證。原創練習用這些區分檢查是否真正理解機制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料

Silberschatz、Galvin、Gagne，*Operating System Concepts*，第九版，第15章，印刷頁657–702；本教材重新推導原創例、圖與練習，不重製原書原題或程式攻擊payload。

RFC5246，§7.4.7.1、7.4.9、8.1與F.1.1.2：歷史TLS1.2 RSA premaster、Finished與forward-secrecy界線。RFC7568，§3：SSL3於2015年禁用。

Microsoft AccessCheck與Mandatory Integrity Control：核對DACL ordered masks與MIC條件；Windows7案例以原書歷史範圍為主，未加入後來AppContainer機制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study